

AI / ML INTERNSHIP PROGRAM

DEVIXO

SOLUTIONS

Task 02 – Build and Compare Machine Learning Models

Version 1.0 approved

Prepared by <Muhammad Hamza Waseem>

Submission Date : 20-09-26

1. Introduction.....	1
2. Objective	2

3. Dataset Name	2
4. Dataset Source	2
5. Dataset Description	2
6. Dataset Size.....	2
7. Features/Columns Description	2
8. Methodology	3
9. Data Loading.....	3
10. Data Cleaning.....	3
11. Conclusion	4
12.Code/Output Screenshots	4
13.Github Rpository Link	7

1. Introduction

In this project, I worked on a real-world classification problem. The main goal was to build and compare multiple machine learning algorithms to see which one predicts student outcomes with the highest accuracy.

2. Objective

The objective of this project is to learn and implement end-to-end machine learning workflows. This includes data preprocessing, building nested pipelines, training multiple models, evaluating their performance using classification reports, and performing hyperparameter tuning using RandomizedSearchCV on a Random Forest model.

3. Dataset Name

Predict Students' Dropout and Academic Success Dataset

4. Dataset Source

Kaggle

5. Dataset Description

This dataset contains various academic, demographic, and socioeconomic factors related to higher education students. The goal is to predict whether a student will Dropout, stay Enrolled, or Graduate.

6. Dataset Size

The dataset contains over 4,000 records and several columns/features.

7. Features/Columns Description

- **Demographic Features:** Includes age at enrollment, gender, nationality, and parental background (such as parents' occupation and education levels).
- **Academic/Socioeconomic Features:** Includes previous qualification (high school grades), admission grade, scholarship status, tuition fees up to date status, and whether the student is a debtor.
- **Academic Performance Features:** Includes units credited, enrolled, evaluated, and approved in both the 1st and 2nd semesters, along with their respective grade averages.
- **Macroeconomic Features:** Includes unemployment rate, inflation rate, and GDP at the time of enrollment.
- **Target Column:** Student's final status (Dropout, Enrolled, or Graduate).

8. Methodology

- To build a clean and structured machine learning system, I used a ColumnTransformer and Pipeline approach in Python
- Separated numerical and categorical columns.
- Applied StandardScaler to normalize numeric values.
- Split the dataset into 80% for training and 20% for testing.

- Trained core models (Logistic Regression, KNN) and later experimented with a Random Forest model using randomized hyperparameter tuning.

9. Data Loading

I loaded the dataset using the pandas library and configured the separator (`sep= ';'`) since the file values were separated by semicolons.

10. Data Cleaning

Checked for missing values and handled them appropriately.

Removed duplicate records to keep the data clean and avoid bias during training.

11. Conclusion

After training and evaluating the models, Random Forest performed the best among the baseline models with an overall test accuracy of **76%**, while Logistic Regression achieved 75%. Furthermore, by applying hyperparameter tuning to the Random Forest model using `RandomizedSearchCV`, the model successfully optimized its parameters (`n_estimators`, `max_depth`, etc.) to find the best possible patterns in the data. Therefore, SVM and the tuned Random Forest are recommended for this prediction task.

12.Code/Output Screenshots

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
```

LOAD DATASET

[+ Code](#)
[+ Text](#)

```
df=pd.read_csv("data.csv" , sep=";")
```

NO OF ROWS AND COLS

```
df.shape
```

```
(4424, 37)
```

FIRST 5 ROWS

```
df.head()
```

	Marital status	Application mode	Application order	Course	Daytime/evening attendance\t	Previous qualification	Previous qualification (grade)	Nacionality	Mother's qualification	Father's qualification	...	Curricular units 2nd sem (credited)	Curricular units 2nd sem (enrolled)
0	1	17	5	171	1	1	122.0	1	19	12	...	0	0
1	1	15	1	9254	1	1	160.0	1	1	3	...	0	0
2	1	1	5	9070	1	1	122.0	1	37	37	...	0	0
3	1	17	2	9773	1	1	122.0	1	38	37	...	0	0
4	2	39	1	8014	0	1	100.0	1	37	38	...	0	0

5 rows x 37 columns

NAME OF ALL COLUMNS

DEVIXO TASK 02

TRAIN/TEST SPLIT

```
standard_scalar_col=df.select_dtypes(include="number").columns
X=df.drop("Target",axis=1)
y=df["Target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

PIPELINES AND COL TRANSFORMERS

```
standard_pipeline=Pipeline(steps=[
    ("standard", StandardScaler())
])
transformer=ColumnTransformer(transformers=[
    ("standard", standard_pipeline, standard_scalar_col)
])
logistic_pipeline=Pipeline(steps=[
    ("model", LogisticRegression())
])
random_pipeline=Pipeline(steps=[
    ("model", RandomForestClassifier())
])
Knn_pipeline=Pipeline(steps=[
    ("model", KNeighborsClassifier())
])
```

```
model1_pipeline=Pipeline(steps=[
    ("transformer", transformer),
    ("logistic_pipeline", logistic_pipeline)
])
model2_pipeline=Pipeline(steps=[
    ("transformer", transformer),
    ("random_pipeline", random_pipeline)
])
model3_pipeline=Pipeline(steps=[
    ("transformer", transformer),
    ("Knn_pipeline", Knn_pipeline)
])
```

TRAIN THE MODELS

```
model1_pipeline.fit(X_train, y_train)
model2_pipeline.fit(X_train, y_train)
model3_pipeline.fit(X_train, y_train)
```

PREDICTIONS

```

6]
0s
y_pred_model1 = model1_pipeline.predict(X_test)
y_pred_model2 = model2_pipeline.predict(X_test)
y_pred_model3 = model3_pipeline.predict(X_test)

```

CLASSIFICATION REPORT

```

7]
0s
▶ y_pred_model1 = model1_pipeline.predict(X_test)
  print(classification_report(y_test, y_pred_model1))
  y_pred_model2 = model2_pipeline.predict(X_test)
  print(classification_report(y_test, y_pred_model2))
  y_pred_model3 = model3_pipeline.predict(X_test)
  print(classification_report(y_test, y_pred_model3))

```

```

...
      precision    recall  f1-score   support

   Dropout      0.83      0.77      0.80      316
   Enrolled      0.45      0.28      0.34      151
   Graduate      0.76      0.91      0.83      418

   accuracy          0.75      885
  macro avg      0.68      0.65      0.66      885
 weighted avg      0.73      0.75      0.74      885

      precision    recall  f1-score   support

   Dropout      0.83      0.76      0.79      316
   Enrolled      0.49      0.30      0.37      151
   Graduate      0.77      0.93      0.84      418

   accuracy          0.76      885
  macro avg      0.70      0.66      0.67      885
 weighted avg      0.74      0.76      0.74      885

      precision    recall  f1-score   support

   Dropout      0.81      0.68      0.74      316
   Enrolled      0.37      0.28      0.32      151
   Graduate      0.72      0.87      0.79      418

   accuracy          0.70      885
  macro avg      0.63      0.61      0.62      885
 weighted avg      0.69      0.70      0.69      885

```

DEVIXO TASK 02

SIMPLE MODEL COMPARISON TABLE:-

Model Name	Accuracy	F1 Score	Advantages	Limitations
Logistic Regression	0.7525	0.7363	Fast and easy to interpret	Struggles with complex patterns
Random Forest	0.7593	0.7429	Handles non-linear patterns well	Can be slow to train on large data
KNN	0.7017	0.6912	Simple implementation	Slow prediction time

Part 5: Conclusion & Recommendation

Recommended Best Model: RANDOM FOREST

Achieved Accuracy: 0.7593

Reason: Based on the classification reports, RANDOM FOREST achieved the highest overall accuracy (0.76) and better balanced precision/recall across classes

RANDOMIZED SEARCH CV

```
[20]  
✓ Os from sklearn.model_selection import RandomizedSearchCV  
from sklearn.ensemble import RandomForestClassifier
```

```
[21]  
✓ Os param_distributions = {  
    'model__n_estimators': [50, 100, 200, 300],  
    'model__max_depth': [10, 20, 30, 40],  
    'model__min_samples_split': [2, 5, 10],  
    'model__min_samples_leaf': [1, 2, 4]  
}  
rf_pipeline = Pipeline(steps=[  
    ("transformers", transformer),  
    ("model", RandomForestClassifier(random_state=42))  
])  
rcv = RandomizedSearchCV(  
    estimator=rf_pipeline,  
    param_distributions=param_distributions,  
    n_iter=5,  
    cv=3,  
    scoring='accuracy',  
    random_state=42,  
    n_jobs=-1  
)
```

BEST PARAMETERS

```
rcv.fit(X_train, y_train)  
print(f"Best params : {rcv.best_params}")  
print(f"Cross-validation : {rcv.best_score}")  
  
best_model = rcv.best_estimator_  
final_acc = best_model.score(X_test, y_test)  
print(f"Test data accuracy: {final_acc}")
```

```
Best params : {'model__n_estimators': 300, 'model__min_samples_split': 5, 'model__min_samples_leaf': 2, 'model__max_depth': 10}  
Cross-validation : 0.7733842718381468  
Test data accuracy: 0.7570621468926554
```

13.Github Rpository Link

<https://github.com/HamzaWaseem2005/ML-REPOSITORY-INTERNSHIP-TASK>